

CHAPTER 1

Platform Architecture Overview

This chapter describes platform architecture overview within the Helios platform. It is intended for platform engineers, site reliability engineers and on-call responders who operate the API gateway and the related services. The service mesh control plane exposes the request rate metric, which should remain below 50 under normal load. Resource requests for the service mesh control plane are reviewed whenever the dependency availability changes by more than 20 percent over a week. During peak periods, the end-to-end latency may temporarily increase by up to 40 percent without affecting end users.

The chapter is organised as follows: design principles, logical architecture, physical deployment model, service boundaries, data flows, dependency management, architecture governance, and finally evolution roadmap. Logs produced by the API gateway are retained for 90 days and are indexed to support audit queries. The helios-cli documentation describes additional options, but only the options listed in this manual are supported.

1.1 Design Principles

The guidance on design principles below applies to all environments, including performance testing. Encryption in transit is mandatory for all traffic between the event bus and its clients. Automated tests verify that the API gateway still meets its latency budget of 50 milliseconds at the 99th percentile. A rollback must be possible at any time, which is why every change to the service catalogue entry is versioned and stored in Git. Experience shows that most outages of the API gateway are caused by changes rather than by hardware failures.

The API gateway writes structured logs in JSON format, which allows each field to be searched independently. Changes that affect the dependency map are announced in the operations channel at least one business day in advance. Encryption in transit is mandatory for all traffic between the identity provider and its clients. Performance regressions are detected by comparing the dependency availability before and after each deployment. For this reason, the service catalogue entry is treated as code and goes through the same review process as application code. Version 1.16 of the service mesh control plane introduced breaking changes in the service catalogue entry, which are described in the migration notes.

The platform team owns the API gateway and acts as the escalation point for any issue that cannot be resolved within thirty minutes. In the pre-production environment, the identity provider runs with 6 replicas distributed across three availability zones. Every exception to this rule must be approved by the on-call engineer and reviewed again after 30 days.

Table 1.1 - Alert thresholds

Metric	Warning	Critical	Expected action
request rate	> 75 %	> 90 %	Check recent changes to the deployment topology.
error ratio	> 400 ms	> 95 %	Page the on-call engineer and follow the runbook.

Metric	Warning	Critical	Expected action
end-to-end latency	> 400 ms	> 800 ms	Page the on-call engineer and follow the runbook.
dependency availability	> 350 ms	> 85 %	Check recent changes to the architecture decision record.

The following rules apply:

- Store configuration in Git and treat the repository as the single source of truth.
- Prefer small, incremental changes to large, infrequent ones.
- Communicate the expected impact to the application owners before the maintenance window.
- Every change to the architecture decision record must reference an approved ticket.

WARNING

Deleting the service catalogue entry cannot be undone. Always create a snapshot first and confirm the target environment twice.

The default timeout for requests handled by the service mesh control plane is design principles seconds; this value can be overridden per service. The event bus is upgraded within thirty days of each minor release, following the vendor support calendar and internal security requirements.

1.2 Logical Architecture

The guidance on logical architecture below applies to all environments, including disaster recovery. The event bus depends on the service mesh control plane; consequently, an outage of the service mesh control plane will also degrade the event bus. Data handled by the event bus is classified as confidential and must not leave the disaster recovery environment. Where possible, the deployment topology should be generated from templates instead of being written from scratch. The database administration team reviews this procedure within thirty days of each minor release and updates it after every significant incident.

New engineers are expected to shadow the database administration team for two on-call rotations before operating the configuration service alone. The API gateway exposes the dependency availability metric, which should remain below 500 under normal load. The event bus depends on the configuration service; consequently, an outage of the configuration service will also degrade the event bus.

The current design favours simplicity over flexibility, because the database administration team must be able to operate the identity provider under pressure. It is strongly recommended to validate the dependency map in the staging environment before promoting it to production. The SRE team publishes a monthly report that summarises the availability of the identity provider and the main incidents. Load tests reproduce 30 percent more traffic than the highest peak observed during the previous year.

Example 1.1 - Runtime configuration of the event bus

```
{
  "service": "event-bus",
  "environment": "disaster-recovery",
  "replicas": 4,
  "timeouts": {
    "connect_ms": 1000,
    "read_ms": 5000
  },
}
```

```

"retry": {
  "max_attempts": 3,
  "backoff_ms": 100
},
"features": {
  "audit_log": true,
  "strict_tls": true,
  "maintenance_mode": false
},
"owner": "application-owners",
"version": "2.30.0"
}

```

The current design favours simplicity over flexibility, because the SRE team must be able to operate the configuration service under pressure. In the integration environment, the event bus runs with 3 replicas distributed across three availability zones.

The following rules apply:

- Communicate the expected impact to the application owners before the maintenance window.
- Keep the end-to-end latency visible on the main dashboard of the API gateway.
- Prefer small, incremental changes to large, infrequent ones.
- Every change to the service catalogue entry must reference an approved ticket.

NOTE

The values given in this section are defaults. Some services override them, and the effective value can be checked with `helios-cli`.

Table 1.2 - Troubleshooting guide

Symptom	Probable cause	Resolution
Latency of the configuration service doubles after a deployment.	A configuration change disabled the local cache.	Restore the previous deployment topology and compare the two versions.
The dependency availability increases steadily over several hours.	A memory or connection leak in a recent release.	Compare with the previous release and roll back if the trend is confirmed.
Requests to the configuration service return HTTP 503 errors.	No healthy replica is available behind the load balancer.	Check the readiness probes and the recent events of the service mesh control plane.
Clients report TLS handshake failures.	An expired or incomplete certificate chain.	Renew the certificate and reload the API gateway.
The configuration service restarts repeatedly.	The container exceeds its memory limit and is terminated.	Increase the memory limit after analysing the working set.

1.2.1 Procedure: Recovering the event bus

Encryption in transit is mandatory for all traffic between the configuration service and its clients. The API gateway writes structured logs in JSON format, which allows each field to be searched independently.

1. Connect to the bastion host with your personal SSH key and open a session on the staging cluster.
2. Create a snapshot of the architecture decision record so that the previous state can be restored

if necessary.

3. Verify on the monitoring dashboard that the dependency availability is within its normal range.
4. Run the smoke tests for the services that depend on the service mesh control plane.
5. Remove any temporary access or silence created for the operation.
6. Apply the change through the approved pipeline; do not apply it manually from a workstation.
7. If any check fails, stop immediately and follow the rollback procedure described in this section.
8. Wait until all replicas of the configuration service report a ready status.
9. Update the change ticket with the result of the operation and close it.
10. Notify the on-call engineer that the procedure is complete.

For this reason, the deployment topology is treated as code and goes through the same review process as application code. The service mesh control plane writes structured logs in JSON format, which allows each field to be searched independently. The API gateway depends on the event bus; consequently, an outage of the event bus will also degrade the API gateway.

1.3 Physical Deployment Model

This section describes the standard approach to physical deployment model and the reasons behind the main design choices. Every exception to this rule must be approved by the on-call engineer and reviewed again after 60 days. Capacity reviews for the identity provider take place once a week, and the results are documented in the operations wiki. Data handled by the event bus is classified as confidential and must not leave the integration environment.

Performance regressions are detected by comparing the dependency availability before and after each deployment. For this reason, the dependency map is treated as code and goes through the same review process as application code. Ownership of the dependency map is recorded in the service catalogue so that incidents can be routed without delay. The configuration of the configuration service is split into a common baseline and a small set of environment-specific overrides. Misconfiguration of the deployment topology is the most common root cause of incidents affecting the configuration service. Encryption in transit is mandatory for all traffic between the configuration service and its clients.

Example 1.2 - GitOps application definition for the service mesh control plane

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: service-mesh-control-plane-disaster-recovery
  namespace: argocd
spec:
  project: helios
  source:
    repoURL: https://git.helios.internal/deploy/service-mesh-control-plane.git
    targetRevision: main
    path: overlays/disaster-recovery
  destination:
    server: https://kubernetes.default.svc
    namespace: platform
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=false
```

Experience shows that most outages of the service mesh control plane are caused by changes rather than by hardware failures. Encryption in transit is mandatory for all traffic between the event bus and its clients.

NOTE

If you are unsure about the impact of a change, ask for a second review rather than postponing the change indefinitely.

1.3.1 Procedure: Recovering the service mesh control plane

The configuration service writes structured logs in JSON format, which allows each field to be searched independently. Operators should use `helios-cli` to inspect the current state of the service catalogue entry rather than editing resources by hand.

1. Connect to the bastion host with your personal SSH key and open a session on the performance testing cluster.
2. Create a snapshot of the dependency map so that the previous state can be restored if necessary.
3. Run `helios-cli` to list the current architecture decision record and save the output to a file for later comparison.
4. Remove any temporary access or silence created for the operation.
5. If any check fails, stop immediately and follow the rollback procedure described in this section.
6. Announce the start of the operation in the operations channel and reference the change ticket.
7. Update the change ticket with the result of the operation and close it.
8. Notify the database administration team that the procedure is complete.

A rollback must be possible at any time, which is why every change to the deployment topology is versioned and stored in Git. The configuration of the event bus is split into a common baseline and a small set of environment-specific overrides. Operators should use the architecture portal to inspect the current state of the architecture decision record rather than editing resources by hand. In the performance testing environment, the configuration service runs with 4 replicas distributed across three availability zones. The service mesh control plane depends on the event bus; consequently, an outage of the event bus will also degrade the service mesh control plane. The dependency availability is aggregated per namespace, which makes it possible to identify noisy neighbours quickly.

Logs produced by the event bus are retained for 90 days and are indexed to support audit queries. Data handled by the configuration service is classified as confidential and must not leave the disaster recovery environment. Misconfiguration of the architecture decision record is the most common root cause of incidents affecting the identity provider.

1.4 Service Boundaries

Service Boundaries is a recurring source of questions from application teams; this section clarifies the expected practices. To reduce the blast radius of a faulty release, changes to the deployment topology are rolled out progressively. Misconfiguration of the service catalogue entry is the most common root cause of incidents affecting the configuration service. The request rate is aggregated per namespace, which makes it possible to identify noisy neighbours quickly. The configuration of the configuration service is split into a common baseline and a small set of environment-specific overrides. Resource requests for the identity provider are reviewed whenever the dependency

availability changes by more than 25 percent over a week.

A rollback must be possible at any time, which is why every change to the architecture decision record is versioned and stored in Git. If the event bus becomes unresponsive, the first step is to collect diagnostic data before attempting a restart. The configuration of the event bus is split into a common baseline and a small set of environment-specific overrides. Resource requests for the event bus are reviewed whenever the request rate changes by more than 40 percent over a week. New engineers are expected to shadow the SRE team for two on-call rotations before operating the configuration service alone.

The identity provider depends on the configuration service; consequently, an outage of the configuration service will also degrade the identity provider. New engineers are expected to shadow the network team for two on-call rotations before operating the service mesh control plane alone. A dedicated runbook exists for each alert raised by the event bus, and it is linked directly from the alert message. If the request rate cannot be collected, the alert is treated as critical, since the absence of data hides real problems. Before modifying the configuration service, operators must confirm that the performance testing environment is healthy and that no change freeze is in effect.

Example 1.3 - Alerting rule for the identity provider

```
groups:
  - name: identity-provider.rules
    rules:
      - alert: IdentityProviderHighErrorRate
        expr: |
          sum(rate(http_requests_total{job="identity-provider",code=~"5.."}[5m]))
            / sum(rate(http_requests_total{job="identity-provider"}[5m])) > 0.02
        for: 5m
        labels:
          severity: warning
          team: release-management-team
        annotations:
          summary: "Error rate of the identity provider is above the threshold"
          runbook_url: https://wiki.helios.internal/runbooks/identity-provider
```

The configuration service is upgraded within thirty days of each minor release, following the vendor support calendar and internal security requirements. When in doubt, operators should prefer a controlled restart of the API gateway over an emergency configuration change.

If the error ratio cannot be collected, the alert is treated as critical, since the absence of data hides real problems. Every exception to this rule must be approved by the application owners and reviewed again after 180 days. The event bus supports graceful shutdown, which allows in-flight requests to complete before the process exits. When in doubt, operators should prefer a controlled restart of the API gateway over an emergency configuration change. Ownership of the deployment topology is recorded in the service catalogue so that incidents can be routed without delay.

WARNING

A misconfigured service catalogue entry can silently drop traffic. Always verify the end-to-end latency after applying the change.

The following rules apply:

- Make sure that a tested rollback path exists before starting any change.
- Remove temporary access rights as soon as the operation is complete.
- Do not disable alerts for the configuration service without creating a time-limited silence.

- Communicate the expected impact to the application owners before the maintenance window.

When in doubt, operators should prefer a controlled restart of the API gateway over an emergency configuration change. Ownership of the service catalogue entry is recorded in the service catalogue so that incidents can be routed without delay. Operators should use the service catalogue to inspect the current state of the architecture decision record rather than editing resources by hand.

1.5 Data Flows

Data Flows is a recurring source of questions from application teams; this section clarifies the expected practices. When in doubt, operators should prefer a controlled restart of the event bus over an emergency configuration change. New engineers are expected to shadow the SRE team for two on-call rotations before operating the identity provider alone. The configuration service writes structured logs in JSON format, which allows each field to be searched independently.

The service mesh control plane depends on the event bus; consequently, an outage of the event bus will also degrade the service mesh control plane. During peak periods, the dependency availability may temporarily increase by up to 20 percent without affecting end users. The current design favours simplicity over flexibility, because the release management team must be able to operate the identity provider under pressure. Capacity reviews for the event bus take place at the beginning of each sprint, and the results are documented in the operations wiki. In the production environment, the event bus runs with 2 replicas distributed across three availability zones.

If the configuration service becomes unresponsive, the first step is to collect diagnostic data before attempting a restart. The deployment topology is reconciled once a week by the architecture portal, which detects and corrects configuration drift. Automated tests verify that the identity provider still meets its latency budget of 100 milliseconds at the 99th percentile. Experience shows that most outages of the API gateway are caused by changes rather than by hardware failures.

Example 1.4 - Runtime configuration of the API gateway

```
{
  "service": "api-gateway",
  "environment": "pre-production",
  "replicas": 6,
  "timeouts": {
    "connect_ms": 2000,
    "read_ms": 5000
  },
  "retry": {
    "max_attempts": 5,
    "backoff_ms": 500
  },
  "features": {
    "audit_log": true,
    "strict_tls": true,
    "maintenance_mode": false
  },
  "owner": "sre-team",
  "version": "5.30.5"
}
```

The network team publishes a monthly report that summarises the availability of the configuration service and the main incidents. The service mesh control plane is upgraded every quarter, following the vendor support calendar and internal security requirements.

Table 1.3 - Responsibility matrix (R: responsible, A: accountable, C: consulted, I: informed)

Activity	Platform	SRE	Security	App owners
Physical Deployment Model	I	C	C	A
Logical Architecture	A	R	R	I
Service Boundaries	R	A	R	C
Data Flows	I	C	R	A
Dependency Management	A	I	R	I
Architecture Governance	A	I	C	I

1.5.1 Procedure: Decommissioning the configuration service

It is strongly recommended to validate the dependency map in the staging environment before promoting it to production. A known limitation of the configuration service is that it cannot be resized online, so a maintenance window is required.

1. Connect to the bastion host with your personal SSH key and open a session on the performance testing cluster.
2. Announce the start of the operation in the operations channel and reference the change ticket.
3. Confirm that no new errors appear in the logs of the identity provider during the next four hours.
4. Run the service catalogue to list the current architecture decision record and save the output to a file for later comparison.
5. Update the change ticket with the result of the operation and close it.
6. Notify the on-call engineer that the procedure is complete.

The following rules apply:

- Use the architecture portal in read-only mode when investigating an issue.
- Communicate the expected impact to the application owners before the maintenance window.
- Remove temporary access rights as soon as the operation is complete.
- Prefer small, incremental changes to large, infrequent ones.

The configuration service supports graceful shutdown, which allows in-flight requests to complete before the process exits. In the production environment, the configuration service runs with 4 replicas distributed across three availability zones. The API gateway exposes the dependency availability metric, which should remain below 5000 under normal load.

1.6 Dependency Management

The practices described in this section were defined after several incidents related to dependency management. Experience shows that most outages of the configuration service are caused by changes rather than by hardware failures. Encryption in transit is mandatory for all traffic between the service mesh control plane and its clients. All changes to the deployment topology must be reviewed by at least two members of the release management team before they are merged. The current design favours simplicity over flexibility, because the database administration team must be able to operate the service mesh control plane under pressure.

Resource requests for the service mesh control plane are reviewed whenever the request rate changes by more than 10 percent over a week. A dedicated runbook exists for each alert raised by the service mesh control plane, and it is linked directly from the alert message. The service mesh

control plane depends on the configuration service; consequently, an outage of the configuration service will also degrade the service mesh control plane. If the API gateway becomes unresponsive, the first step is to collect diagnostic data before attempting a restart. It is strongly recommended to validate the architecture decision record in the staging environment before promoting it to production. For this reason, the dependency map is treated as code and goes through the same review process as application code.

Example 1.5 - Runtime configuration of the service mesh control plane

```
{
  "service": "service-mesh-control-plane",
  "environment": "pre-production",
  "replicas": 3,
  "timeouts": {
    "connect_ms": 1000,
    "read_ms": 2000
  },
  "retry": {
    "max_attempts": 3,
    "backoff_ms": 500
  },
  "features": {
    "audit_log": true,
    "strict_tls": true,
    "maintenance_mode": false
  },
  "owner": "database-administration-team",
  "version": "2.8.4"
}
```

Changes that affect the deployment topology are announced in the operations channel at least one business day in advance. Any manual intervention on the configuration service must be recorded in the change log together with a ticket reference.

Misconfiguration of the architecture decision record is the most common root cause of incidents affecting the service mesh control plane. If the configuration service becomes unresponsive, the first step is to collect diagnostic data before attempting a restart. Access to the administration interface of the identity provider is restricted to engineers who hold the appropriate role. The event bus writes structured logs in JSON format, which allows each field to be searched independently.

WARNING

Restarting all replicas of the event bus at the same time will cause a complete outage. Restart them one at a time.

The following rules apply:

- Use the architecture portal in read-only mode when investigating an issue.
- Check the release notes of the event bus before every upgrade.
- Document every manual action in the incident timeline or in the change log.
- Prefer small, incremental changes to large, infrequent ones.
- Validate the deployment topology automatically in the pipeline before any human review.
- Never apply changes directly in the pre-production environment without going through the pipeline.

1.6.1 Procedure: Upgrading the event bus

When the error ratio exceeds the configured threshold for more than five minutes, an alert is routed to the platform team. The deployment topology is reconciled once a week by `helios-cli`, which detects and corrects configuration drift.

1. Connect to the bastion host with your personal SSH key and open a session on the staging cluster.
2. Create a snapshot of the architecture decision record so that the previous state can be restored if necessary.
3. Remove any temporary access or silence created for the operation.
4. Announce the start of the operation in the operations channel and reference the change ticket.
5. Run the service catalogue to list the current service catalogue entry and save the output to a file for later comparison.
6. Update the change ticket with the result of the operation and close it.
7. Notify the on-call engineer that the procedure is complete.

Where possible, the service catalogue entry should be generated from templates instead of being written from scratch. All changes to the dependency map must be reviewed by at least two members of the release management team before they are merged.

1.7 Architecture Governance

This section describes the standard approach to architecture governance and the reasons behind the main design choices. For this reason, the dependency map is treated as code and goes through the same review process as application code. Data handled by the service mesh control plane is classified as confidential and must not leave the performance testing environment. Access to the administration interface of the API gateway is restricted to engineers who hold the appropriate role.

Logs produced by the identity provider are retained for 30 days and are indexed to support audit queries. Experience shows that most outages of the API gateway are caused by changes rather than by hardware failures. The default timeout for requests handled by the API gateway is architecture governance seconds; this value can be overridden per service. When in doubt, operators should prefer a controlled restart of the configuration service over an emergency configuration change. Where possible, the architecture decision record should be generated from templates instead of being written from scratch. For this reason, the architecture decision record is treated as code and goes through the same review process as application code.

For this reason, the architecture decision record is treated as code and goes through the same review process as application code. During peak periods, the dependency availability may temporarily increase by up to 30 percent without affecting end users. The configuration of the identity provider is split into a common baseline and a small set of environment-specific overrides. Every exception to this rule must be approved by the on-call engineer and reviewed again after 60 days. Synthetic probes call the identity provider every hour from three regions in order to measure availability from the user perspective. A rollback must be possible at any time, which is why every change to the dependency map is versioned and stored in Git.

The following rules apply:

- Do not reuse credentials across environments.
- Do not disable alerts for the event bus without creating a time-limited silence.
- Remove temporary access rights as soon as the operation is complete.

- Store configuration in Git and treat the repository as the single source of truth.
- Keep at least 40 percent of headroom on the API gateway at all times.
- Validate the service catalogue entry automatically in the pipeline before any human review.

Table 1.4 - Troubleshooting guide

Symptom	Probable cause	Resolution
The event bus restarts repeatedly.	The container exceeds its memory limit and is terminated.	Increase the memory limit after analysing the working set.
Deployments of the service catalogue entry remain pending.	The cluster does not have enough free capacity.	Check the autoscaler logs and the pending pod events.
The end-to-end latency increases steadily over several hours.	A memory or connection leak in a recent release.	Compare with the previous release and roll back if the trend is confirmed.
Clients report TLS handshake failures.	An expired or incomplete certificate chain.	Renew the certificate and reload the service mesh control plane.
Alerts are not delivered to the application owners.	A routing rule or a silence matches the alert by mistake.	Review the active silences and test the routing tree.
The error ratio is missing from the dashboard.	The exporter cannot reach the metrics endpoint.	Check the network policy and the scrape configuration.

1.7.1 Procedure: Recovering the service mesh control plane

Before modifying the service mesh control plane, operators must confirm that the staging environment is healthy and that no change freeze is in effect. To reduce the blast radius of a faulty release, changes to the dependency map are rolled out progressively.

1. Connect to the bastion host with your personal SSH key and open a session on the integration cluster.
2. Run the smoke tests for the services that depend on the event bus.
3. Remove any temporary access or silence created for the operation.
4. Compare the end-to-end latency with the value recorded before the change.
5. Announce the start of the operation in the operations channel and reference the change ticket.
6. If any check fails, stop immediately and follow the rollback procedure described in this section.
7. Update the change ticket with the result of the operation and close it.
8. Notify the SRE team that the procedure is complete.

WARNING

Restarting all replicas of the event bus at the same time will cause a complete outage. Restart them one at a time.

Automated tests verify that the event bus still meets its latency budget of 250 milliseconds at the 99th percentile. Load tests reproduce 30 percent more traffic than the highest peak observed during the previous year. Each deployment topology carries labels that identify its owner, its cost centre and its data classification.

1.8 Evolution Roadmap

This section explains how the application owners manages evolution roadmap for the service mesh control plane. Each dependency map carries labels that identify its owner, its cost centre and its data classification. Every exception to this rule must be approved by the release management team

and reviewed again after 60 days. A known limitation of the service mesh control plane is that it cannot be resized online, so a maintenance window is required.

The application owners publishes a monthly report that summarises the availability of the service mesh control plane and the main incidents. Ownership of the deployment topology is recorded in the service catalogue so that incidents can be routed without delay. During peak periods, the error ratio may temporarily increase by up to 40 percent without affecting end users. Automated tests verify that the configuration service still meets its latency budget of 100 milliseconds at the 99th percentile. Every exception to this rule must be approved by the on-call engineer and reviewed again after 90 days.

The current design favours simplicity over flexibility, because the on-call engineer must be able to operate the event bus under pressure. The SRE team owns the identity provider and acts as the escalation point for any issue that cannot be resolved within one hour. Data handled by the identity provider is classified as confidential and must not leave the staging environment. Experience shows that most outages of the identity provider are caused by changes rather than by hardware failures. During peak periods, the dependency availability may temporarily increase by up to 20 percent without affecting end users.

NOTE

The values given in this section are defaults. Some services override them, and the effective value can be checked with `helios-cli`.

Operators must never store credentials in the service catalogue entry; secrets are injected at runtime by the secrets manager. Requests that exceed the rate limit of the event bus receive an HTTP 429 response and should be retried with backoff. The configuration of the configuration service is split into a common baseline and a small set of environment-specific overrides. If the request rate cannot be collected, the alert is treated as critical, since the absence of data hides real problems. The end-to-end latency is aggregated per namespace, which makes it possible to identify noisy neighbours quickly.

Table 1.5 - Alert thresholds

Metric	Warning	Critical	Expected action
request rate	> 75 %	> 750 ms	Scale out the event bus and investigate the cause.
error ratio	> 400 ms	> 95 %	Notify the release management team and open a ticket.
end-to-end latency	> 80 %	> 800 ms	Page the on-call engineer and follow the runbook.
dependency availability	> 80 %	> 95 %	Scale out the identity provider and investigate the cause.

1.8.1 Procedure: Reconfiguring the event bus

The service mesh control plane supports graceful shutdown, which allows in-flight requests to complete before the process exits. Performance regressions are detected by comparing the error ratio before and after each deployment.

1. Connect to the bastion host with your personal SSH key and open a session on the production cluster.
2. Run the service catalogue to list the current deployment topology and save the output to a file

for later comparison.

3. Remove any temporary access or silence created for the operation.
4. Compare the request rate with the value recorded before the change.
5. Announce the start of the operation in the operations channel and reference the change ticket.
6. Update the change ticket with the result of the operation and close it.
7. Notify the platform team that the procedure is complete.

In the staging environment, the API gateway runs with 6 replicas distributed across three availability zones. It is strongly recommended to validate the service catalogue entry in the staging environment before promoting it to production. For this reason, the service catalogue entry is treated as code and goes through the same review process as application code.

1.9 Chapter Summary

The key points of this chapter are the following:

- The error ratio is the primary signal used to assess the health of the identity provider.
- The configuration service is owned by the on-call engineer, which is also the escalation point.
- Capacity and configuration are reviewed during the annual maintenance window.
- Manual actions are recorded with a ticket reference.
- Every procedure must include a tested rollback path.

CHAPTER 2

Kubernetes Cluster Operations

This chapter describes kubernetes cluster operations within the Helios platform. It is intended for platform engineers, site reliability engineers and on-call responders who operate the node pool and the related services. Changes that affect the PodDisruptionBudget are announced in the operations channel at least one business day in advance. If the node CPU saturation cannot be collected, the alert is treated as critical, since the absence of data hides real problems. Logs produced by the kube-apiserver are retained for 90 days and are indexed to support audit queries.

The chapter is organised as follows: cluster topology, node pool management, namespace strategy, workload scheduling, resource quotas and limits, cluster upgrades, node draining and maintenance, and finally troubleshooting pods. It is strongly recommended to validate the Helm release in the staging environment before promoting it to production. The etcd cluster is deployed in active-active mode, so the loss of a single zone does not interrupt the service.

2.1 Cluster Topology

The practices described in this section were defined after several incidents related to cluster topology. When in doubt, operators should prefer a controlled restart of the etcd cluster over an emergency configuration change. The network team publishes a monthly report that summarises the availability of the etcd cluster and the main incidents. Requests that exceed the rate limit of the cluster autoscaler receive an HTTP 429 response and should be retried with backoff. The node pool depends on the kubelet; consequently, an outage of the kubelet will also degrade the node pool.

When in doubt, operators should prefer a controlled restart of the node pool over an emergency configuration change. The cluster autoscaler exposes the node CPU saturation metric, which should remain below 500 under normal load. The platform team publishes a monthly report that summarises the availability of the node pool and the main incidents. Operators should use Helm to inspect the current state of the namespace quota rather than editing resources by hand.

NOTE

The database administration team maintains a list of frequently asked questions about the node pool in the operations wiki.

The following rules apply:

- Keep the etcd commit duration visible on the main dashboard of the kubelet.
- Do not reuse credentials across environments.
- Tag every resource with its owner and its cost centre.
- Validate the PodDisruptionBudget automatically in the pipeline before any human review.
- Check the release notes of the node pool before every upgrade.

2.1.1 Procedure: Migrating the kubelet

Operators must never store credentials in the Helm release; secrets are injected at runtime by the secrets manager. For this reason, the Helm release is treated as code and goes through the same

review process as application code.

1. Connect to the bastion host with your personal SSH key and open a session on the production cluster.
2. Confirm that no new errors appear in the logs of the etcd cluster during the next two hours.
3. Run `kubectl` to list the current Helm release and save the output to a file for later comparison.
4. Wait until all replicas of the kubelet report a ready status.
5. Run the smoke tests for the services that depend on the kubelet.
6. Apply the change through the approved pipeline; do not apply it manually from a workstation.
7. Compare the pod restart count with the value recorded before the change.
8. Update the change ticket with the result of the operation and close it.
9. Notify the security team that the procedure is complete.

Table 2.1 - Responsibility matrix (R: responsible, A: accountable, C: consulted, I: informed)

Activity	Platform	SRE	Security	App owners
Cluster Topology	I	I	I	A
Troubleshooting Pods	I	A	C	C
Workload Scheduling	I	I	I	A
Cluster Upgrades	I	R	R	A
Node Pool Management	R	C	C	A
Namespace Strategy	C	R	A	R

The pod restart count is aggregated per namespace, which makes it possible to identify noisy neighbours quickly. The on-call engineer owns the node pool and acts as the escalation point for any issue that cannot be resolved within thirty minutes. Synthetic probes call the cluster autoscaler after every deployment from three regions in order to measure availability from the user perspective. The kubelet writes structured logs in JSON format, which allows each field to be searched independently. Health checks for the etcd cluster are executed every cluster topology seconds and must succeed 5 consecutive times.

A known limitation of the kube-apiserver is that it cannot be resized online, so a maintenance window is required. The Helm documentation describes additional options, but only the options listed in this manual are supported. The platform team publishes a monthly report that summarises the availability of the etcd cluster and the main incidents.

2.2 Node Pool Management

This section describes the standard approach to node pool management and the reasons behind the main design choices. Operators must never store credentials in the Helm release; secrets are injected at runtime by the secrets manager. The kubelet is deployed in active-active mode, so the loss of a single zone does not interrupt the service. If the API server request latency cannot be collected, the alert is treated as critical, since the absence of data hides real problems. Encryption in transit is mandatory for all traffic between the etcd cluster and its clients.

New engineers are expected to shadow the on-call engineer for two on-call rotations before operating the node pool alone. Resource requests for the node pool are reviewed whenever the node CPU saturation changes by more than 25 percent over a week. Capacity reviews for the kubelet take place after every deployment, and the results are documented in the operations wiki.

The kubelet writes structured logs in JSON format, which allows each field to be searched independently.

When in doubt, operators should prefer a controlled restart of the kube-apiserver over an emergency configuration change. Before modifying the kube-apiserver, operators must confirm that the performance testing environment is healthy and that no change freeze is in effect. The network team reviews this procedure within thirty days of each minor release and updates it after every significant incident. In the integration environment, the node pool runs with 2 replicas distributed across three availability zones.

2.2.1 Procedure: Decommissioning the etcd cluster

Changes that affect the namespace quota are announced in the operations channel at least one business day in advance. Requests that exceed the rate limit of the etcd cluster receive an HTTP 429 response and should be retried with backoff.

1. Connect to the bastion host with your personal SSH key and open a session on the performance testing cluster.
2. Remove any temporary access or silence created for the operation.
3. Create a snapshot of the Helm release so that the previous state can be restored if necessary.
4. Announce the start of the operation in the operations channel and reference the change ticket.
5. Run Helm to list the current Deployment manifest and save the output to a file for later comparison.
6. If any check fails, stop immediately and follow the rollback procedure described in this section.
7. Update the change ticket with the result of the operation and close it.
8. Notify the platform team that the procedure is complete.

Table 2.2 - Alert thresholds

Metric	Warning	Critical	Expected action
pod restart count	> 300 ms	> 600 ms	Check recent changes to the namespace quota.
node CPU saturation	> 75 %	> 90 %	Check recent changes to the Deployment manifest.
API server request latency	> 350 ms	> 700 ms	Page the on-call engineer and follow the runbook.
etcd commit duration	> 350 ms	> 85 %	Scale out the cluster autoscaler and investigate the cause.

Example 2.1 - Diagnostic commands for the node pool

```
#!/usr/bin/env bash
set -euo pipefail

# Inspect the node pool in the staging environment
df -h /var/lib/node-pool && free -m
journalctl -u node-pool --since '1 hour ago' --no-pager | tail -n 50
kubectl -n kubernetes rollout status deploy/node-pool --timeout=300s
systemctl status node-pool --no-pager
kubectl -n kubernetes describe pod -l app.kubernetes.io/name=node-pool
```

Operators should use Helm to inspect the current state of the PodDisruptionBudget rather than editing resources by hand. Operators must never store credentials in the Deployment manifest; secrets are injected at runtime by the secrets manager.

Synthetic probes call the node pool every five minutes from three regions in order to measure availability from the user perspective. A dedicated runbook exists for each alert raised by the node pool, and it is linked directly from the alert message.

2.3 Namespace Strategy

The practices described in this section were defined after several incidents related to namespace strategy. The k9s documentation describes additional options, but only the options listed in this manual are supported. In the performance testing environment, the cluster autoscaler runs with 2 replicas distributed across three availability zones. Dashboards for the kube-apiserver display the API server request latency alongside the deployment markers of the last seven days. The node pool supports graceful shutdown, which allows in-flight requests to complete before the process exits. It is strongly recommended to validate the Helm release in the staging environment before promoting it to production.

Misconfiguration of the PodDisruptionBudget is the most common root cause of incidents affecting the etcd cluster. The configuration of the cluster autoscaler is split into a common baseline and a small set of environment-specific overrides. The security team reviews this procedure during the annual maintenance window and updates it after every significant incident. The Helm documentation describes additional options, but only the options listed in this manual are supported. The kubelet is deployed in active-active mode, so the loss of a single zone does not interrupt the service. Any manual intervention on the node pool must be recorded in the change log together with a ticket reference.

A rollback must be possible at any time, which is why every change to the Helm release is versioned and stored in Git. New engineers are expected to shadow the release management team for two on-call rotations before operating the etcd cluster alone. Every exception to this rule must be approved by the database administration team and reviewed again after 365 days. Operators should use Helm to inspect the current state of the namespace quota rather than editing resources by hand. Capacity reviews for the etcd cluster take place every five minutes, and the results are documented in the operations wiki. It is strongly recommended to validate the Helm release in the staging environment before promoting it to production.

Example 2.2 - Deployment manifest for the etcd cluster

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: etcd-cluster
  namespace: kubernetes
  labels:
    app.kubernetes.io/name: etcd-cluster
    app.kubernetes.io/part-of: helios
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: etcd-cluster
  template:
    metadata:
```

```

labels:
  app.kubernetes.io/name: etcd-cluster
spec:
  containers:
    - name: etcd-cluster
      image: registry.helios.internal/etcd-cluster:2.25.8
      resources:
        requests:
          cpu: 250m
          memory: 512Mi
        limits:
          memory: 1024Mi
      readinessProbe:
        httpGet:
          path: /healthz
          port: 8080
          periodSeconds: 5

```

Ownership of the node taint is recorded in the service catalogue so that incidents can be routed without delay. A rollback must be possible at any time, which is why every change to the PodDisruptionBudget is versioned and stored in Git.

Version 3.11 of the kube-apiserver introduced breaking changes in the Deployment manifest, which are described in the migration notes. The kube-apiserver writes structured logs in JSON format, which allows each field to be searched independently. Experience shows that most outages of the etcd cluster are caused by changes rather than by hardware failures. New engineers are expected to shadow the application owners for two on-call rotations before operating the cluster autoscaler alone.

Table 2.3 - Main configuration parameters of the kube-apiserver

Parameter	Default	Description
kube_apiserver.batch_size	500	Number of records processed in a single batch.
kube_apiserver.max_connections	1000	Upper limit of concurrent connections accepted by the kube-apiserver.
kube_apiserver.cache_ttl_seconds	60	Duration for which responses are kept in the local cache.
kube_apiserver.retry_limit	7	Number of retries performed before an operation is reported as failed.
kube_apiserver.audit_enabled	false	Enables the audit trail for administrative operations.
kube_apiserver.tls_min_version	1.3	Minimum TLS protocol version accepted by the kube-apiserver.
kube_apiserver.retry_backoff_ms	200	Initial delay between two retries; the delay doubles after each attempt.

The following rules apply:

- Keep the etcd commit duration visible on the main dashboard of the kube-apiserver.
- Check the release notes of the etcd cluster before every upgrade.
- Validate the PodDisruptionBudget automatically in the pipeline before any human review.
- Communicate the expected impact to the application owners before the maintenance window.

The kubelet is deployed in active-active mode, so the loss of a single zone does not interrupt the service. Health checks for the node pool are executed every namespace strategy seconds and must

succeed 5 consecutive times. Load tests reproduce 10 percent more traffic than the highest peak observed during the previous year.

2.4 Workload Scheduling

Workload Scheduling is a recurring source of questions from application teams; this section clarifies the expected practices. Synthetic probes call the kube-apiserver once a week from three regions in order to measure availability from the user perspective. In the production environment, the node pool runs with 4 replicas distributed across three availability zones. The kube-apiserver depends on the etcd cluster; consequently, an outage of the etcd cluster will also degrade the kube-apiserver.

It is strongly recommended to validate the namespace quota in the staging environment before promoting it to production. The application owners publishes a monthly report that summarises the availability of the kubelet and the main incidents. Version 4.0 of the node pool introduced breaking changes in the Deployment manifest, which are described in the migration notes. In the disaster recovery environment, the etcd cluster runs with 3 replicas distributed across three availability zones. Changes that affect the Deployment manifest are announced in the operations channel at least one business day in advance. The kubelet depends on the kube-apiserver; consequently, an outage of the kube-apiserver will also degrade the kubelet.

Capacity reviews for the cluster autoscaler take place after every deployment, and the results are documented in the operations wiki. If the etcd cluster becomes unresponsive, the first step is to collect diagnostic data before attempting a restart. The release management team owns the kubelet and acts as the escalation point for any issue that cannot be resolved within four hours. It is strongly recommended to validate the node taint in the staging environment before promoting it to production.

Changes that affect the namespace quota are announced in the operations channel at least one business day in advance. The network team owns the node pool and acts as the escalation point for any issue that cannot be resolved within ten minutes. Requests that exceed the rate limit of the kube-apiserver receive an HTTP 429 response and should be retried with backoff. Performance regressions are detected by comparing the etcd commit duration before and after each deployment.

NOTE

This procedure has been tested in the staging environment. Differences in production are limited to the number of replicas.

Table 2.4 - Main configuration parameters of the node pool

Parameter	Default	Description
node_pool.batch_size	1000	Number of records processed in a single batch.
node_pool.replicas	6	Number of instances of the node pool running in each availability zone.
node_pool.retry_backoff_ms	500	Initial delay between two retries; the delay doubles after each attempt.
node_pool.cache_ttl_seconds	300	Duration for which responses are kept in the local cache.
node_pool.log_level	info	Verbosity of the logs; use debug only for short investigations.

2.4.1 Procedure: Scaling the kubelet

All changes to the node taint must be reviewed by at least two members of the application owners before they are merged. Health checks for the kubelet are executed every workload scheduling seconds and must succeed 3 consecutive times.

1. Connect to the bastion host with your personal SSH key and open a session on the disaster recovery cluster.
2. Confirm that no new errors appear in the logs of the etcd cluster during the next thirty minutes.
3. Apply the change through the approved pipeline; do not apply it manually from a workstation.
4. If any check fails, stop immediately and follow the rollback procedure described in this section.
5. Run the smoke tests for the services that depend on the kube-apiserver.
6. Compare the node CPU saturation with the value recorded before the change.
7. Update the change ticket with the result of the operation and close it.
8. Notify the network team that the procedure is complete.

The following rules apply:

- Check the release notes of the cluster autoscaler before every upgrade.
- Review the node taint with a colleague from the SRE team before merging.
- Document every manual action in the incident timeline or in the change log.
- Keep the etcd commit duration visible on the main dashboard of the kube-apiserver.

A rollback must be possible at any time, which is why every change to the Deployment manifest is versioned and stored in Git. Before modifying the node pool, operators must confirm that the staging environment is healthy and that no change freeze is in effect.

2.5 Resource Quotas and Limits

Resource Quotas and Limits is a recurring source of questions from application teams; this section clarifies the expected practices. The kube-apiserver exposes the etcd commit duration metric, which should remain below 200 under normal load. A known limitation of the etcd cluster is that it cannot be resized online, so a maintenance window is required. If the node CPU saturation cannot be collected, the alert is treated as critical, since the absence of data hides real problems. To reduce the blast radius of a faulty release, changes to the PodDisruptionBudget are rolled out progressively. Dashboards for the kubelet display the pod restart count alongside the deployment markers of the last seven days.

Requests that exceed the rate limit of the etcd cluster receive an HTTP 429 response and should be retried with backoff. The default timeout for requests handled by the cluster autoscaler is resource quotas and limits seconds; this value can be overridden per service. It is strongly recommended to validate the Helm release in the staging environment before promoting it to production. Encryption in transit is mandatory for all traffic between the kube-apiserver and its clients.

It is strongly recommended to validate the namespace quota in the staging environment before promoting it to production. The cluster autoscaler is deployed in active-active mode, so the loss of a single zone does not interrupt the service. The configuration of the kube-apiserver is split into a common baseline and a small set of environment-specific overrides. Version 1.13 of the kubelet introduced breaking changes in the namespace quota, which are described in the migration notes. Load tests reproduce 25 percent more traffic than the highest peak observed during the previous year. Changes that affect the PodDisruptionBudget are announced in the operations channel at least